



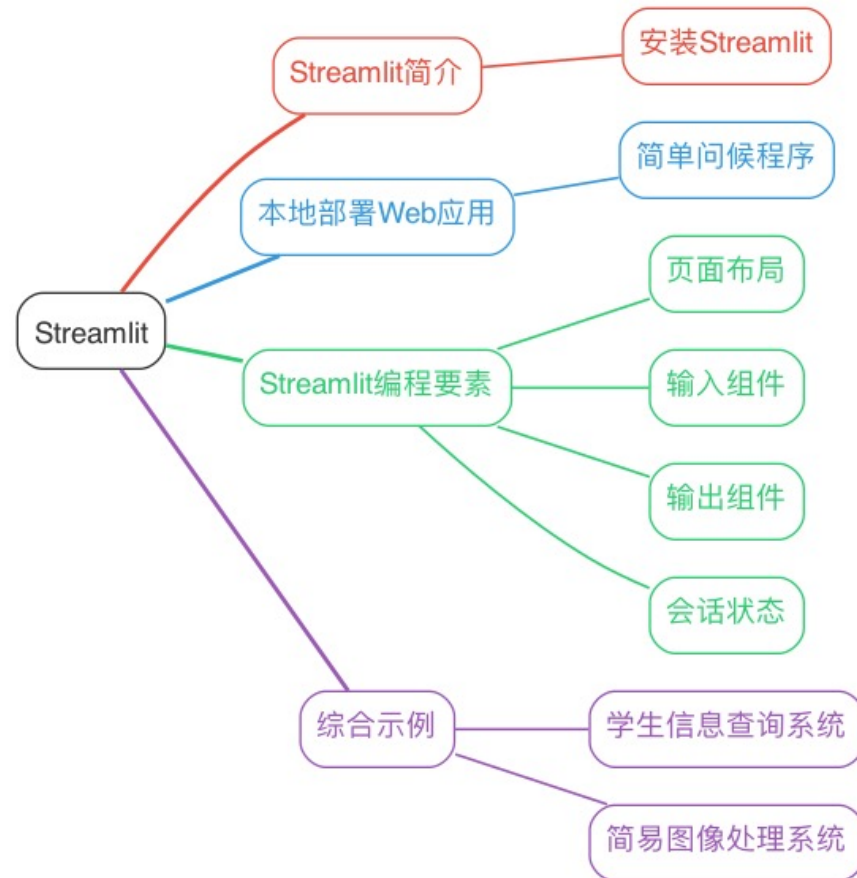
《实用Python编程》

第12讲: Streamlit交互式数据分析与可视化

实用Python编程 教研组



提纲



Streamlit简介

- 用于快速创建数据分析与可视化Web应用的Python工具包
 - 几行Python代码就能构建界面美观、交互性强的网页
 - 无需使用HTML、CSS、JavaScript等网页编程语言

```
import streamlit as st
import pandas as pd
import numpy as np

st.write("Streamlit supports a wide range of data visualizations, i

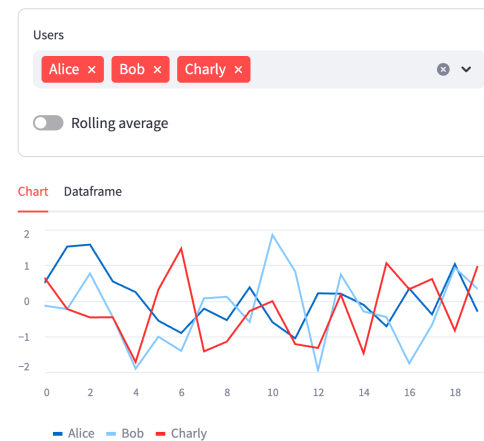
all_users = ["Alice", "Bob", "Charly"]
with st.container(border=True):
    users = st.multiselect("Users", all_users, default=all_users)
    rolling_average = st.toggle("Rolling average")

np.random.seed(42)
data = pd.DataFrame(np.random.randn(20, len(users)), columns=users)
if rolling_average:
    data = data.rolling(7).mean().dropna()

tab1, tab2 = st.tabs(["Chart", "Dataframe"])
tab1.line_chart(data, height=250)
tab2.dataframe(data, height=250, use_container_width=True)
```

Share

Streamlit supports a wide range of data visualizations, including [Plotly](#), [Altair](#), and [Bokeh charts](#). And with over 20 input widgets, you can easily make your data interactive!



用Python进行
前端开发与UI设计!

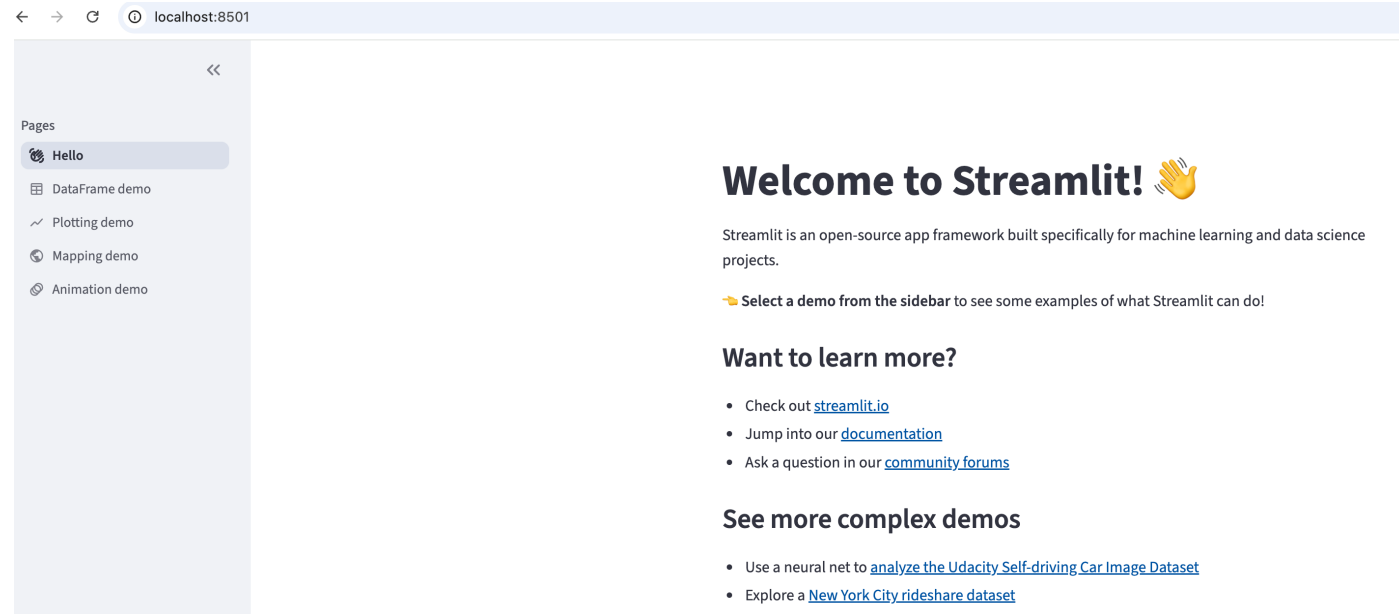
<https://streamlit.io/>



安装Streamlit

- 安装指令 `pip install streamlit`
- 安装完成后，在命令行中执行指令 `streamlit hello`

若浏览器自动打开如右侧所示的欢迎页面，则表明Streamlit 已就绪





本地部署Web应用

- 利用Streamlit实现一个简单的问候程序

我的第一个 Streamlit 应用

Hello, Streamlit!

请输入你的名字

张三

你好，张三！

用户在文本输入框中输入一个名字后，页面会在下方自动显示问候语

本地部署Web应用

- 将示例代码保存为一个以.py为扩展名的Python文件 (**st-hello.py**)
- 在命令行界面中 (Streamlit不能在Jupyter Notebook中运行)
 - 先通过cd命令将工作路径切换到**st-hello.py**所在目录
 - 再执行指令 `streamlit run st-hello.py`

```
1 import streamlit as st
```

```
3 st.title("我的第一个 Streamlit 应用")
```

```
4 st.write("Hello, Streamlit!")
```

```
5 name = st.text_input("请输入你的名字")
```

```
6 if name:
```

```
7     st.success(f"你好, {name}! ")
```

输出组件

我的第一个 Streamlit 应用

Hello, Streamlit!

请输入你的名字

输入组件

你好, 张三!

输出组件



Streamlit编程要素

- 在前端开发与UI设计中，**页面布局**、**输入组件**和**输出组件**是构建用户界面的三大核心要素
 - 页面布局
 - 负责骨架与空间安排，控制页面结构与元素排列，为内容提供容器和占位
 - 输入组件
 - 负责接收用户数据，捕获用户的交互行为与数据输入
 - 输出组件
 - 向用户展示数据、状态或计算结果
- 会话状态`st.session_state`：与用户交互的记忆体



页面布局

- 常用的页面布局函数

- `st.set_page_config`
- `st.columns`
- `st.sidebar`
- `st.tabs`
- `st.container`

页面设置

多列

侧边栏

多标签页

容器

页面布局

- `st.set_page_config` 设置页面

控制内容的展示方式。"centered"（默认）会将内容固定在页面中央，而 "wide" 模式则会让内容铺满整个屏幕，适合展示宽表格或复杂图表。

```
st.set_page_config(layout="centered",  
                  page_title="我的数据分析程序")
```

设置浏览器标签页上显示的文字

页面布局

- `st.columns` 把页面水平分割成多个列容器
 - 使用 `with col:` 语句块

```
1 cols = st.columns(3)
2 headers = ['冲击区间', '稳妥区间', '保底区间']
3 colleges = ['A大学', 'B大学', 'C大学']
4 subjects = ['D专业', 'E专业', 'F专业']
5 for i,col in enumerate(cols):
6     with col:
7         st.header(headers[i])
8         st.subheader(f'{colleges[i]}')
9         st.write(f'{subjects[i]}')
```

with用于限定代码第7-9行所添加的内容均属于当前列

cols[0]

冲击区间

A大学

D专业

cols[1]

稳妥区间

B大学

E专业

cols[2]

保底区间

C大学

F专业

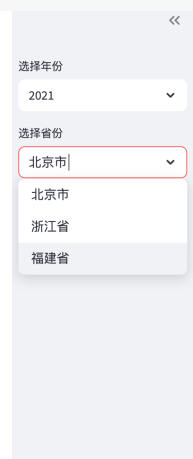
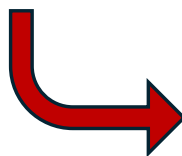
页面布局

- `st.sidebar` 创建侧边栏，用于将导航菜单（筛选条件、参数控制等）固定在页面左侧
 - 使用 `with st.sidebar:` 语句块，将所有侧边栏内容包裹其中

```
with st.sidebar:
```

```
    year = st.selectbox("选择年份:", [2021, 2022, 2023])
```

```
    prov = st.selectbox("选择省份:", ["北京市", "浙江省", "福建省"])
```



2021年-中国人民大学-北京市录取分数线

	专业名称	最高分	最低分
0	财政学类	671	670
1	法学	678	674
2	工商管理类	675	668
3	国际政治	672	666
4	金融学类	678	672
5	经济学类	676	667
6	理科试验班	679	667
7	理科试验班 (基础学科类)	676	667
8	人力资源管理	668	666
9	人文科学试验班	677	670



课堂练习-6-3-1

- 如图所示，将侧边栏中选择的年份、省份分别显示在右侧页面的第1列和第2列中

选择年份:	第1列	第2列
2021	2021	
选择省份:		
北京市		北京市

提示：列名用输出组件 `st.header`，即`st.header(f'{第i列}')`

页面布局

- `st.tabs`用于创建标签页，将页面内容分组成多个可切换的板块

```
import streamlit as st

# 创建三个标签页
tab1, tab2, tab3 = st.tabs(["首页", "数据图表", "关于"])

# 第一个标签页的内容
with tab1:
    st.header("欢迎来到首页")
    st.write("这里是第一个标签页的主要内容。")

# 第二个标签页的内容
with tab2:
    st.header("数据分析")
    st.line_chart([1, 5, 2, 6, 4])

# 第三个标签页的内容
with tab3:
    st.header("关于我们")
    st.success("这是一个使用 st.tabs 的简单示例。")
```



首页 数据图表 关于

欢迎来到首页

这里是第一个标签页的主要内容。

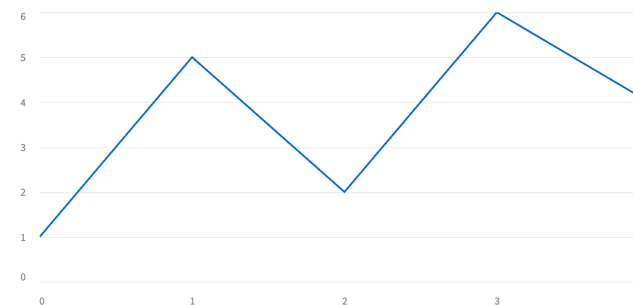
首页 数据图表 关于

关于我们

这是一个使用 `st.tabs` 的简单示例。

首页 数据图表 关于

数据分析





页面布局

- `st.container` 用于精细控制内容布局、动态更新部分界面等
 - Streamlit 默认按代码从上到下的顺序渲染界面。
 - 使用 `st.container` 可以先占位，后续再往里填入内容。

```
import streamlit as st

# 1. 先在页面顶部创建一个容器占位
top_container = st.container()

# 2. 在页面中部渲染普通组件
st.write("--- 页面中部区域 ---")
user_input = st.text_input("请输入内容: ")

# 3. 满足条件时，将内容倒灌回顶部的容器中
if user_input:
    top_container.success(f"你在下方输入了: {user_input}")
else:
    top_container.info("等待下方输入...")
```



等待下方输入...

你在下方输入了: hello

--- 页面中部区域 ---

--- 页面中部区域 ---

请输入内容:

请输入内容:

hello



输入组件

捕获用户的交互行为与数据输入

- `st.text_input` # 输入单行短文本
- `st.text_area` # 输入多行长文本
- `st.number_input` # 输入数值
- `st.slider` # 滑块式输入数值
- `st.button` # 按钮
- `st.form_submit_button` # 表单数据批量提交按钮
- `st.radio` # 单选按钮
- `st.checkbox` # 复选框
- `st.selectbox` # 下拉选择
- `st.multiselect` # 多选
- `st.file_uploader` # 上传文件

输入组件

- 编写用于采集用户信息的表单（示例代码6.32）

```
1 st.title("用户信息收集")
2 with st.form("user_info_form"):
3     col1, col2 = st.columns(2)
4     with col1:
5         name = st.text_input("姓名*")
6         age = st.number_input("年龄", 0, 120, 25)
7         email = st.text_input("邮箱*")
8     with col2:
9         gender = st.radio("性别", ["男", "女", "其他"], horizontal=True)
10        interests = st.multiselect("兴趣", ["阅读", "运动", "音乐", "旅行", "编程"])
11        newsletter = st.checkbox("订阅通讯")
12    submitted = st.form_submit_button("提交")
13    if submitted:
14        if not name or not email:
15            st.error("请填写必填字段（带*的）")
16        else:
17            st.success("信息提交成功！")
18            st.json({"姓名": name, "年龄": age, "邮箱": email, "性别": gender, "兴趣": interests, "订阅通
                讯": newsletter})
```

st.form是一个容器，用于限定批处理交互的边界

用户信息收集



```
{
  "姓名": "张三"
  "年龄": 25
  "邮箱": "zhangsan@email.com"
  "性别": "男"
  "兴趣": [
    0: "运动"
    1: "旅行"
  ]
  "订阅通讯": true
}
```



课堂练习-6-3-2

- 将用户信息采集表单中的年龄改为滑块式 (st.slider) 输入

用户信息收集

姓名*

性别 ☒ 男 ☐ 女 ☐ 其他

年龄 - +

兴趣

邮箱*

☐ 订阅通讯



用户信息收集

姓名*

性别 ☒ 男 ☐ 女 ☐ 其他

年龄

兴趣

邮箱*

☐ 订阅通讯

输出组件

根据输出内容的格式，Streamlit输出组件可分为纯文本类、结构化数据类、图表类、图像视频类等

主标题

章节标题

子章节标题

纯文本

markdown文本支持加粗、斜体等

代码块

```
def hello():
    print('Hello Streamlit!')
```

latex 公式

$$f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b = \sum_{i=1}^d w_i x_i + b$$

DataFrame展示

	姓名	年龄	薪资
0	张三	25	8000
1	李四	30	12000
2	王五	28	9500

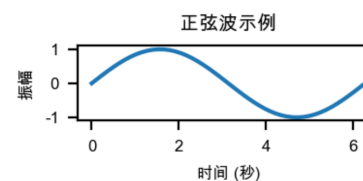
JSON数据展示

```
{
  "name": "Streamlit",
  "version": "1.52.1"
}
```

内置折线图



Matplotlib 图表



展示网络图片



Streamlit Logo

展示由NumPy生成的随机噪点图



随机噪点图



输出组件

- 在页面中展示纯文本数据

```
st.title("主标题")
st.header("章节标题")
st.subheader("子章节标题")
st.text("纯文本")
st.markdown("markdown文本支持**加粗**、*斜体*等")
st.info('代码块')
st.code("def hello():\n    print('Hello Streamlit!)", language="python")
st.info('latex 公式')
st.latex(r"f(\mathbf{x})=\mathbf{w}^T \mathbf{x} + b=\sum_{i=1}^d w_i x_i + b")
```



主标题

章节标题

子章节标题

纯文本

markdown文本支持加粗、斜体等

代码块

```
def hello():
    print('Hello Streamlit!')
```

latex 公式

$$f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b = \sum_{i=1}^d w_i x_i + b$$

输出组件

- 在页面中展示结构化数据

```
df = pd.DataFrame({
    '姓名': ['张三', '李四', '王五'],
    '年龄': [25, 30, 28],
    '薪资': [8000, 12000, 9500]
})
st.info("DataFrame展示")
st.dataframe(df)

st.info("JSON数据展示")
st.json({"name": "Streamlit", "version": f"{st.__version__}"})
```



DataFrame展示

	姓名	年龄	薪资
0	张三	25	8000
1	李四	30	12000
2	王五	28	9500

JSON数据展示

```
{
  "name": "Streamlit"
  "version": "1.52.1"
}
```

输出组件

- 在页面中展示图表

```
st.info("内置折线图")
chart_data = pd.DataFrame(np.random.randn(10, 3), columns=['A', 'B', 'C'])
st.line_chart(chart_data)

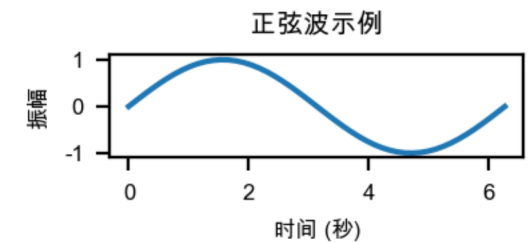
st.info("Matplotlib 图表")
fig, ax = plt.subplots(figsize=(2,0.5))
x = np.linspace(0, 2*3.14, 50)
y = np.sin(x)
ax.plot(x, y)
ax.set_xlabel('时间 (秒)')
ax.set_ylabel('振幅')
ax.set_title('正弦波示例')
st.pyplot(fig) # 将 Matplotlib 的 Figure 对象传递给 Streamlit
```



内置折线图



Matplotlib 图表



输出组件

在页面中展示图像、视频

- `st.image`支持本地路径、URL、PIL Image对象、NumPy数组

```
st.info("展示网络图片")
st.image("https://streamlit.io/images/brand/
streamlit-logo-primary-colormark-darktext.svg",
caption="Streamlit Logo", width=300)

st.info("展示由NumPy生成的随机噪点图")
st.image(np.random.randint(0, 256, (200, 300, 3)), caption="随机噪点图")
```



展示网络图片



Streamlit Logo

展示由NumPy生成的随机噪点图



随机噪点图

- `st.video`支持MP4, WebM等格式及YouTube链接

```
st.video('https://www.youtube.com/watch?v=J0Aq44Pze-w')
```



会话状态

- 一个网站需要支持多个用户的并发访问。会话 (Session) 是网站为每个用户提供的 (临时) 专属交互环境
- 每次用户交互 (点击按钮、改输入框等) 都会触发重新运行 (rerun) 事件
- 会话状态 (st.session_state) 像词典一样记录上一次交互后的系统状态

```
1 if 'counter' not in st.session_state:
2     st.session_state.counter = 0
3 if st.button('点我'):
4     st.session_state.counter += 1
5 st.info(f'按钮被点了{st.session_state.counter}次')
```

设置一个
计数器

点我

按钮被点了0次

点我

按钮被点了1次

点我

按钮被点了5次

示例代码：利用会话状态记录按钮被点击的次数



会话状态

- 看一下这个程序的功能是什么？有没有问题？

```
1  if 'c1' not in st.session_state:
2      st.session_state.c1 = 0
3  if 'c2' not in st.session_state:
4      st.session_state.c2 = 0
5  if st.button('第一个按钮'):
6      st.session_state.c1 += 1
7      if st.button('第二个按钮'):
8          st.session_state.c2 += 1
9  st.info(f'第一个按钮被点了{st.session_state.c1}次，第二个按钮被点了{st.session_state.c2}次')
```



学生信息查询系统

- 根据用户输入的学号返回相应记录

学生信息查询系统

	姓名	性别	出生日期	学号	所属学院	入学年份	籍贯	GPA	总学分
0	张伟	男	1999-05-12	2018010101	计算机学院	2018	江苏省南京市	3.6	142
1	李静	女	2000-08-23	2019020203	经济学院	2019	浙江省杭州市	3.8	126
2	王明	男	2001-02-14	2020030305	外国语学院	2020	广东省广州市	3.2	108
3	刘芳	女	1999-11-05	2018040407	法学院	2018	湖北省武汉市	3.7	156
4	陈浩	男	2000-07-19	2019050509	医学院	2019	四川省成都市	3.9	132
5	杨雪	女	2001-04-30	2020060611	艺术学院	2020	陕西省西安市	3.4	112
6	赵阳	男	1999-09-17	2018070713	机械工程学院	2018	山东省济南市	3.5	148
7	周琳	女	2000-12-08	2019080815	数学学院	2019	湖南省长沙市	3.8	128
8	吴强	男	2001-03-21	2020090917	物理学院	2020	河南省郑州市	3.1	98
9	郑婷婷	女	1999-06-25	2018101019	化学学院	2018	福建省福州市	3.6	152

页面分为两列，一列用于显示学生记录表，
另一列用于容纳查询板块

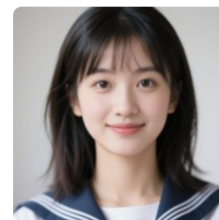
请输入学号:

2019080815

查询

查询结果

	姓名	性别	出生日期	学号	所属学院	入学年份	籍贯	GPA	总学分
7	周琳	女	2000-12-08	2019080815	数学学院	2019	湖南省长沙市	3.8	128





学生信息查询系统

- 完整代码
- 基于Pandas的数据加载、查询
- 基于Streamlit的数据可视化、用户交互

```
1 student_file = 'pybook-data/ch3/rawdata.txt'
2 photo_dir = 'pybook-data/ch3/portraits'
3 columns = '姓名,性别,出生日期,学号,所属学院,入学年份,籍贯,GPA,总学分'.split(',')
4 df = pd.read_csv(student_file, header=None, names=columns)
5 st.set_page_config(layout="wide")
6 st.title('学生信息查询系统')
7 col1, col2 = st.columns(2, gap="large")
8 with col1:
9     st.dataframe(df)
10 with col2:
11     with st.form("query_form"):
12         query = st.text_input("请输入学号:")
13         submitted = st.form_submit_button("查询")
14         if submitted:
15             result = df[df['学号'].astype('str')==query]
16             if not result.empty:
17                 st.info('查询结果')
18                 st.dataframe(result)
19                 st.image(os.path.join(photo_dir,f'{query}.png'), width=200)
20             else:
21                 st.warning(f'学号{query}不存在')
```

布尔索引

简易图像处理系统

- 用户通过滑动条实时调整参数

选择一张图片上传

Drag and drop file here
Limit 200MB per file • JPG, J...

Browse files

2020060611....
1.5MB

Gamma 对比度校正

原始图像信息: 宽度 1328 px, 高度 1328 px, 颜色模式 RGB

调整参数

Gamma 值

2.15

原图 ($\gamma = 1.0$)

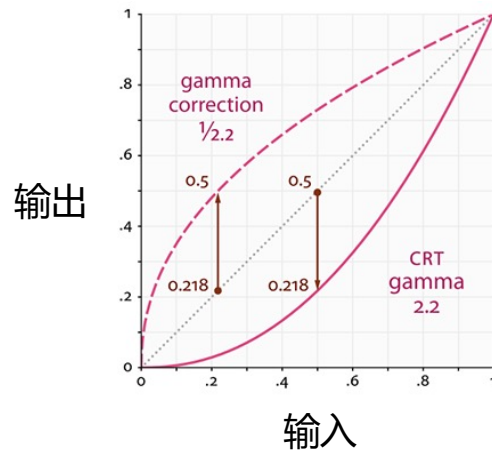


调整后 ($\gamma = 2.15$)



简易图像处理系统

- Gamma校正



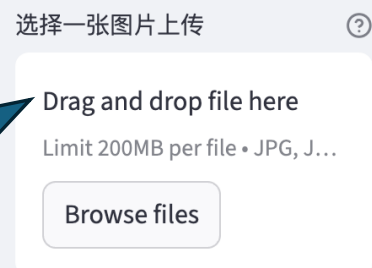
```
import numpy as np
from PIL import Image
from skimage import exposure, img_as_float, img_as_ubyte

def gamma_correction(rgb_image, gamma):
    img_array = np.array(rgb_image) # 转换为 numpy 数组
    img_float = img_as_float(img_array) # 转换为 float [0, 1] 范围
    img_adjusted = exposure.adjust_gamma(img_float, gamma=gamma) # Gamma 校正
    img_ubyte = img_as_ubyte(img_adjusted) # 转回 uint8 [0, 255] 范围
    return Image.fromarray(img_ubyte)
```

简易图像处理系统

- 使用文件上传组件 `st.file_uploader` 上传图片

```
with st.sidebar:
    uploaded_file =
    st.file_uploader("选择一张
    图片上传", type=["jpg",
    "jpeg", "png"], help="支持
    JPG, JPEG, PNG格式")
```



Gamma 对比度校正

请先在侧边栏上传一张图片

简易图像处理系统

- 主程序中需要区分“未上传图片”、“图片已上传”这两种状态

```
with st.sidebar:
    uploaded_file = st.file_uploader("选择一张图片上传", type=["jpg", "jpeg", "png"],
                                     help="支持 JPG, JPEG, PNG格式")

if uploaded_file is None: # 未上传图片时的提示
    st.info("请先在侧边栏上传一张图片")
else:
    image = Image.open(uploaded_file) # 使用 PIL 读取图片
    st.success(f"原始图像信息: 宽度 {image.width} px, 高度 {image.height} px, 颜色模式 {image.mode}")
    (width, height) = (image.width // 4, image.height // 4)
    image = image.resize((width, height))
    if image.mode != 'RGB': # 转换为RGB, 以确保格式统一
        image = image.convert('RGB')

    st.subheader("🖼️ 调整参数")
    gamma = st.slider("Gamma 值", min_value=0.1, max_value=3.0, value=1.0, step=0.05)

    adjusted_image = gamma_correction(image, gamma)
```